inference compute into reference-free supervision. Our focus differs: we use parallelism within **PDR** to explore and distill at inference time, and analyze compute explicitly under fixed B_{seq} while varying B_{total}.

Multi-trace selection and aggregation. Confidence-aware test-time scaling generates multiple traces in parallel and filters or re-weights them with model-internal confidence, improving the accuracy–compute trade-off without extra training (Fu et al., 2025). Another line trains an *aggregator* to select or combine solutions using RL rather than majority vote or reward-model ranking (Zhao et al., 2025). In contrast, we cast inference itself as *round-wise operators* (**SR**, **PDR**), with aggregation as one of the meta-cognitive abilities necessary to improve the performance, and propose **PDR** RL to reduce the train-inference mismatch. Zheng et al. (2025) propose *Parallel-R1*, an RL framework that instills parallel thinking through a progressive curriculum: supervised fine-tuning on easier prompts to bootstrap the behavior, followed by RL on harder problems. On math benchmarks (MATH, AMC23, AIME), Parallel-R1 improves over a sequential-thinking RL baseline and shows that parallel thinking functions as a mid-training exploration scaffold before aiding verification in later stages.

Structured search beyond single chains. Prompting schemes structure exploration explicitly: *Tree of Thoughts* (ToT) expands and evaluates branches of reasoning (Yao et al., 2023); *Graph of Thoughts* generalizes to arbitrary thought graphs (Besta et al., 2024); *Least-to-Most* decomposes problems into subproblems solved in sequence (Zhou et al., 2022). These methods typically grow tokens with breadth/depth or rely on long contexts to carry intermediate state. In contrast, **PDR** concentrates exploration within a round, then distills to a bounded $C^{(r)}$, preventing unbounded context growth.

Multi-agent debate and compressed debate. Debate-style methods have multiple LLM "agents" propose answers and iteratively read and critique one another, often improving robustness (Du et al., 2023a). Our *Parallel-Distill-Refine (***PDR***)* can be seen as a *compressed* debate: treat a round's diverse drafts as agent outputs, but instead of replaying full transcripts, distill them into a bounded $C^{(r)}$ that conditions the next round. This preserves cross-agent scrutiny while controlling per-call context and both B_{seq} and B_{total}.

Compact summaries vs. persistent memory. Agent systems often add external memory or retrieval to persist context across sessions/tasks; thought buffers and memory-augmented agents exemplify this direction (e.g., Buffer-of-Thoughts) (Yang et al., 2024). We instead use *non-persistent*, round-wise summaries. This choice keeps prompts short and reduces long-context failure modes (Liu et al., 2023). Concurrently, Wu et al. (2025) introduce ReSum, a web-agent paradigm that periodically summarizes growing interaction histories into compact states and pairs this with ReSum-GRPO so agents learn summary-conditioned reasoning. Our work differs in focusing on multiple inference-time operators instead of summary (**SR**, **PDR**), alongside operator-consistent RL aligned to the round-wise interface. Didolkar et al. (2025) introduce *Metacognitive Reuse*, extracting recurring reasoning into concise, named behaviors that reduce token usage and can be distilled via SFT. This is complementary to our within-instance **PDR**: rather than compressing cross-instance procedures, we parallelize and distill drafts per instance.

Adaptive multi-turn training. Unary-feedback multi-turn RL ("try again") trains models to revise across rounds improving multi-turn accuracy with minimal supervision (Liu et al., 2025a). *Exploratory Iteration* (ExIt) leverages the recurrent structure of self-improvement by performing multi-step refinement at test time while training emphasizes the most informative single-step iterations (Jiang et al., 2025). **SR** is similar to "try again" iterative interface.

RL with on-policy tree search. Hou et al. (2025) introduce *TreeRL*, which integrates on-policy tree search into RL for LLM reasoning, improving exploration and providing dense, process-level rewards (Xie et al., 2024) compared to independent chain sampling with outcome-only supervision. TreeRL thus exemplifies a *training-time, search-augmented* improvement operator. In contrast, our operator-consistent RL trains in the same iterative interface used at inference: we unroll a single round (akin to a shallow tree expansion) but optimize with outcome supervision without deep tree search. A promising direction is to incorporate process-level rewards into this operator-consistent setting and study their impact on the test-time performance of round-wise operators (**SR**, **PDR**).

3 LLMs as Improvement Operators

3.1 Problem setting and notation

We consider tasks x (e.g., a math problem) and aim to produce a high-quality final artifact s_{final} (solution, proof, or program) under a given token budget. Let $\mathcal{M}_\theta$ denote a (frozen or trainable) LLM used as an *improvement operator*. Given a current artifact s_t (single generation or set of generations) and a compact textual workspace C_t, the model proposes a refinement:

$$s_{t+1} \leftarrow \mathcal{M}_\theta(x, s_t, C_t). \tag{1}$$

The workspace C_t is a bounded summary ($|C_t| \leq \kappa$ tokens) meant to capture agreements, contradictions, intermediate results, and open subgoals.

Read-write-compress cycle. Each step (i) reads the current workspace C_t, (ii) writes a refined artifact s_{t+1} via $\mathcal{M}_\theta$, and (iii) compresses back into a bounded workspace for the next step using a synthesis operator $\mathcal{D}$:

$$C_{t+1} \leftarrow \mathcal{D}(x, s_{t+1}), \qquad |C_{t+1}| \leq \kappa. \tag{2}$$

Token budgets. We evaluate every method under two budgets:

$$\mathrm{B}_{\text{seq}} = \sum_{c \in \mathcal{P}} \big(\text{in}_c + \text{out}_c\big) \qquad \text{(latency proxy; tokens along the accepted path)}, \tag{3}$$

$$\mathrm{B}_{\text{total}} = \sum_{c=1}^{C} \big(\text{in}_c + \text{out}_c\big) \qquad \text{(compute/cost proxy; all calls, including discarded branches)}. \tag{4}$$

Here $c = 1, \ldots, C$ indexes all model calls (prompts, candidate generations, and distillation/summary updates); in_c and out_c are the input and output tokens for call c; and $\mathcal{P} \subseteq \{1, \ldots, C\}$ is the final accepted path. We report accuracy as a function of both axes and match baselines per axis (e.g., equal B_{seq} for latency-controlled comparisons).

3.2 Operator Instantiations

We study two short-context iterative refinement pipelines.

3.2.1 Sequential refinement (SR; depth over a single candidate).

We set $C_t \equiv \varnothing$ for all t and iteratively improve a single artifact for R rounds:

$$s_{t+1} \leftarrow \mathcal{M}_\theta(x, s_t, \varnothing), \quad t = 0, \ldots, R-1, \qquad s_{\text{final}} = s_R. \tag{5}$$

SR with compact workspace. In **SR**, no explicit workspace is provided. We also evaluate a variant that inserts an error analysis step between rounds: rather than directly refining the previous answer, the model first identifies and explains flaws in the current solution, then generates a revised solution. These notes act as a transient, local workspace at each round.

3.2.2 Parallel-Distill-Refine (PDR; round-wise workspace)

We do not maintain a persistent memory. Instead, for rounds $r = 1, \ldots, R$; we sample M_r drafts (Parallel) conditioned on the current bounded summary, then re-synthesize (Distill) a fresh bounded summary for the next round:

$$\text{(Parallel)} \quad S^{(r)} = \big\{\, s_i^{(r)} \leftarrow \mathcal{M}_\theta(x, C^{(r-1)}) \,\big\}_{i=1}^{M_r}, \qquad C^{(0)} = \varnothing, \tag{6}$$

$$\text{(Distill)} \quad C^{(r)} \leftarrow \mathcal{D}\big(x, S^{(r)}\big), \qquad |C^{(r)}| \leq \kappa. \tag{7}$$

We enforce single generation in last round $M_R = 1$; which is returned as s_{final}. The summary is round-wise and non-persistent: earlier text is not replayed, preventing growth in per-call context.

Why a round-wise summary? Replay of all prior attempts scales linearly with steps and reintroduces long-context failure modes. Re-synthesizing $C^{(r)}$ from the current drafts keeps the memory *bounded* ($|C^{(r)}| \leq \kappa$) and focuses each round on the most recent and informative evidence.

Constructing the compact summary $C^{(r)}$. We consider several practical instantiations of the distillation operator $\mathcal{D}$, all obeying $|C^{(r)}| \leq \kappa$:

- **Global summary:** Produce a single shared $C^{(r)}$ that captures agreements, contradictions, derived facts, unresolved subgoals, and next actions. This emphasizes verification and comparison while retiring stale or contradicted information.
- **Extractive top-k evidence (shared):** Instead of free-form text, select the k solutions from $S^{(r)}$ as the workspace itself, trading compression for higher fidelity to the best evidence.
- **Random-k / bootstrapped workspaces:** For the next round, construct multiple small workspaces by randomly sampling k solutions per generation. This injects diversity and mitigates premature consensus while keeping each workspace small.

Budgets. Tokens used for **Parallel**, **Distill**, and **Refine** contribute to $\mathrm{B}_{\text{total}}$. The reported latency B_{seq} only counts the tokens on the accepted generate→distill→refine path for the final output.

3.3 Operator-Consistent Training

The previous sections treat $\mathcal{M}_\theta$ as frozen and rely purely on prompting/orchestration. We now make sure training is consistent with deployment/inference by optimizing the model under the same short-context, iterative interface used at test time.

Motivation. Most RL for reasoning LLMs optimizes a single, long chain-of-thought trajectory. If inference instead runs multiple short passes with a compact workspace C, this creates a train-test mismatch. We remove this mismatch by mixing two training modes: (i) standard long-trace optimization, and (ii) *operator rollouts* that execute the generate→distill→refine interface under short contexts.

Base Algorithm. For the baseline RL, we use the CISPO objective from Minimax-M1 (Li et al., 2025). For a given prompt x, the generator $\pi(\cdot \mid \theta_{\text{old}})$ generates G rollouts $\{o_{i=1}^{G}\}$ using the old policy θ_{old}. Automated checkers like sympy (Meurer et al., 2017) or math-verify[2] are used to assign scalar rewards r_i (± 1) to each of the rollouts. CISPO combines the group-normalized advantage from GRPO (Shao et al., 2024) with REINFORCE (Williams, 1992) to achieve the following objective:

$$\mathcal{J}_{\text{CISPO}}(\theta) = \mathop{\mathbb{E}}_{x\sim\mathcal{D},\{o_i\}_{i=1}^{G}\sim\pi(\cdot|x;\theta_{\text{old}})} \left[\frac{1}{\sum_{i=1}^{G}|o_i|} \sum_{i=1}^{G}\sum_{t=1}^{|o_i|} \mathrm{sg}(r_{i,t}(\theta)) A_i \log(\pi(o_{i,t}|x, o_{i,<t};\theta)) \right] \tag{8}$$

where $A_i = \frac{r_i - \mathrm{mean}(\{r\}_{j=1}^{G})}{\mathrm{std}(\{r\}_{j=1}^{G})}$ is the advantage, sg is the stop-gradient operation, and $r_{i,t}(\theta)$ is computed using the asymmetric clipping from Yu et al. (2025) as follows:

$$r_{i,t} = \mathrm{clip}\left(\frac{\pi(o_i \mid x, o_{i,<t};\theta)}{\pi(o_i \mid x, o_{i,<t};\theta_{\text{old}})}, 1-\epsilon^-, 1+\epsilon^+ \right) \tag{9}$$

where $\frac{\pi(o_i|x,o_{i,<t};\theta)}{\pi(o_i|x,o_{i,<t};\theta_{\text{old}})}$ is the importance-sampling (IS) weight. Additionally, we add an SFT loss (negative log-likelihood) similar to Seed et al. (2025) on rollouts which lead to positive rewards. The final training objective becomes:

$$\mathcal{J}(\theta) = \mathcal{J}_{\text{CISPO}}(\theta) + \alpha \cdot \mathcal{J}_{\text{SFT}}(\theta) \tag{10}$$

[2] https://github.com/huggingface/Math-Verify